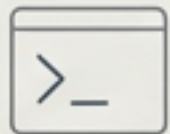
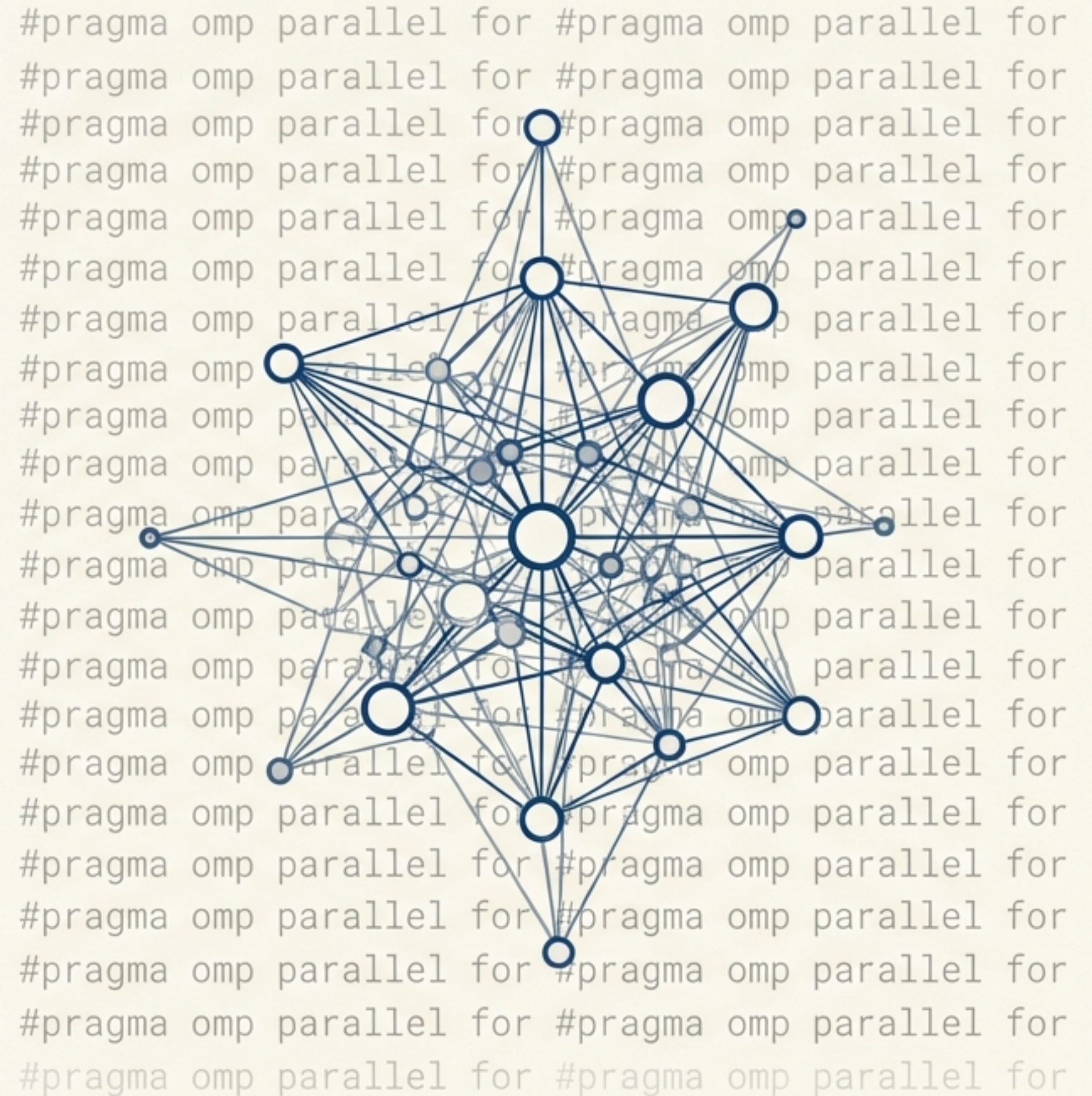


Computación Paralela en Física con OpenMP

De la teoría secuencial a la simulación de alto desempeño.



Sugerencia para el profesor: Iniciar la clase ejecutando un script de Hola Mundo desde 8 hilos simultáneos para demostrar la división del trabajo en vivo.



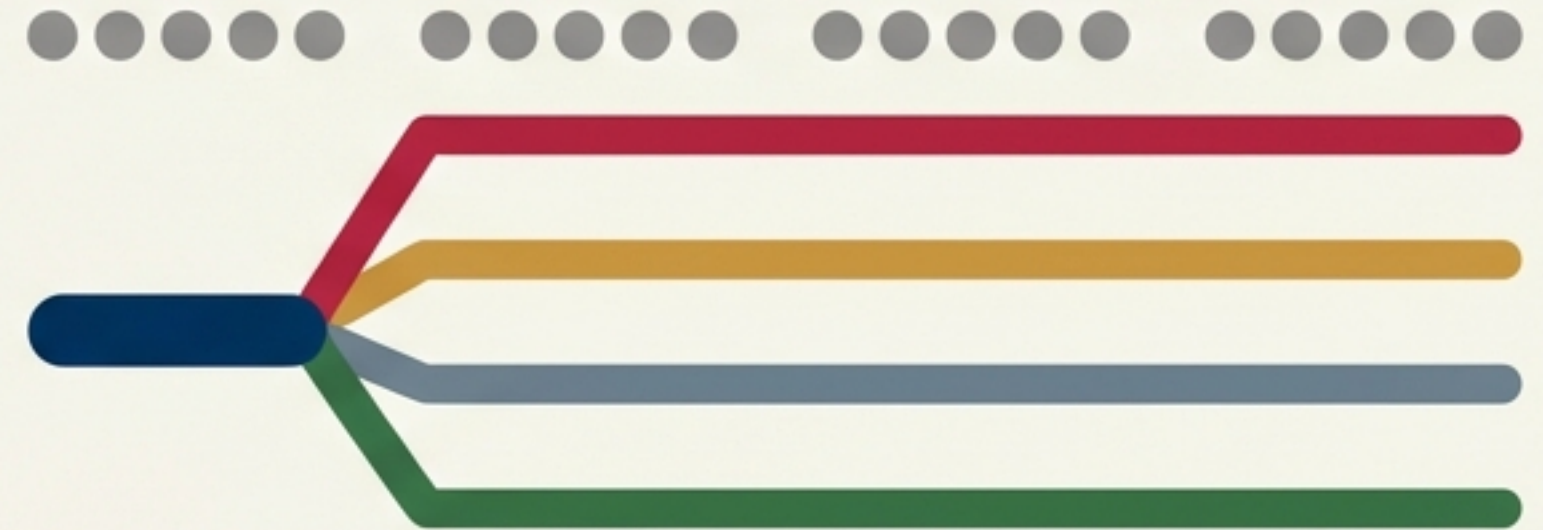
El cuello de botella de la física secuencial

Los sistemas físicos reales requieren calcular millones de interacciones.
La programación secuencial limita este cálculo a un solo núcleo del procesador.

Secuencial

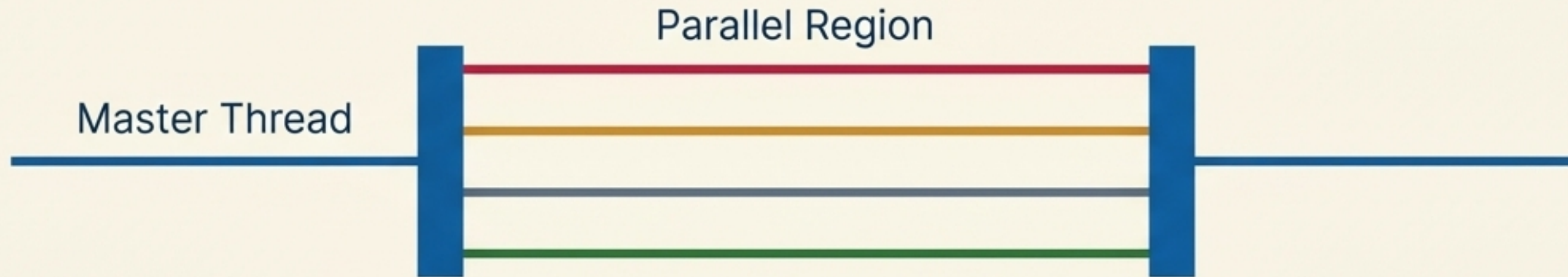


Paralelo



La Solución: Dividir el trabajo espacial, estadístico o iterativo entre múltiples núcleos simultáneamente.

OpenMP: El superpoder de C++ para paralelizar



```
g++ simulacion.cpp -fopenmp -O2  
./a.out
```

Directiva Mágica

`#pragma omp parallel` for divide automáticamente un bucle (loop) entre todos los núcleos disponibles del sistema.

Control de Hilos

Se ajusta con `export OMP_NUM_THREADS=8` en la terminal o `omp_set_num_threads(8);` en el código.

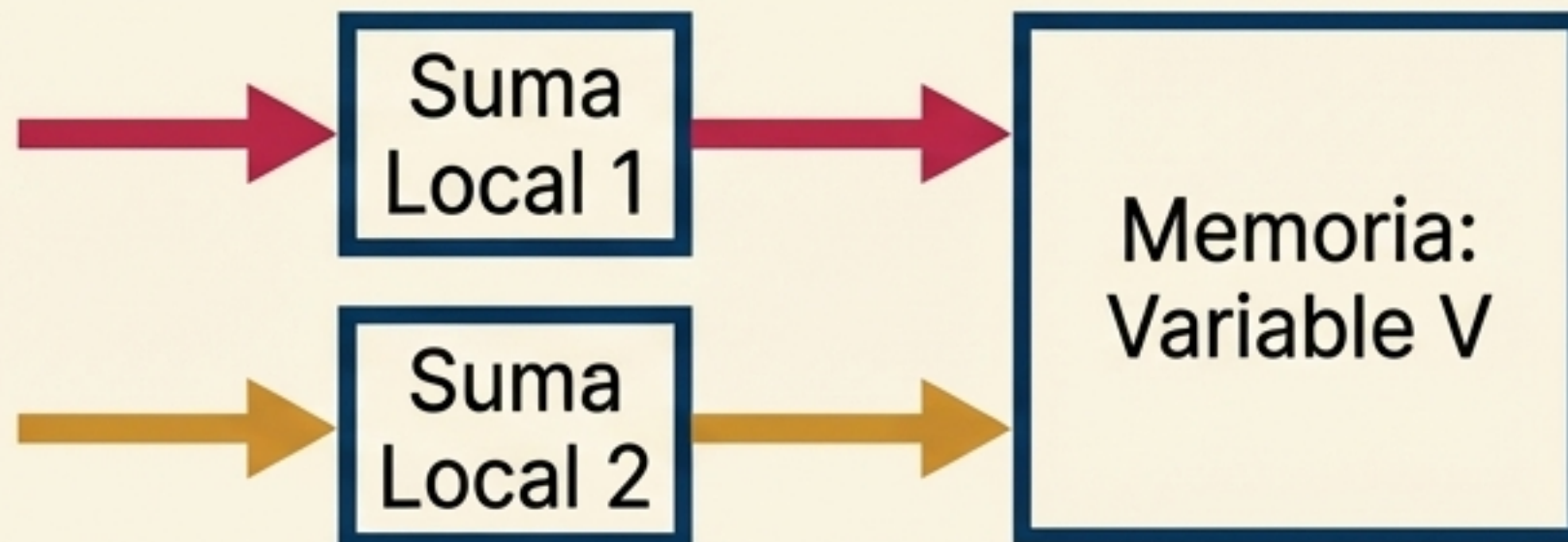
Anatomía de una colisión: Race Conditions y Reducción

El Peligro: Race Condition



Ocurre cuando múltiples hilos intentan leer y escribir en la misma variable de memoria simultáneamente, corrompiendo el resultado.

La Solución: Reducción

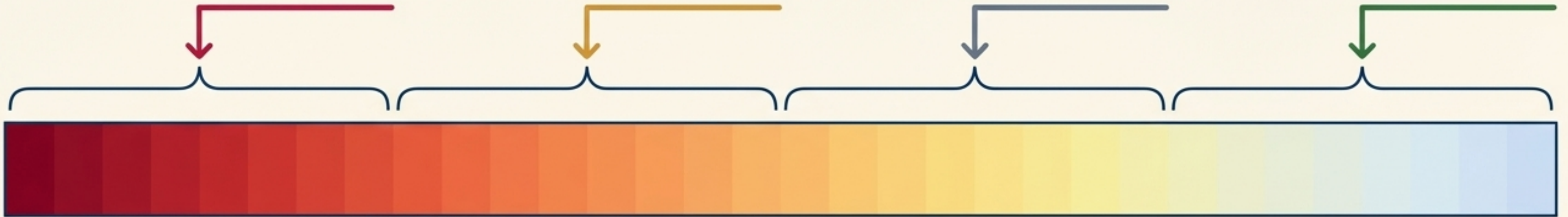


La cláusula `reduction(+:V)` ordena a OpenMP que cada hilo **calcule una suma parcial** en privado y luego **combine los resultados** de forma **segura** paso a paso.

Paradigma 1: Paralelismo Espacial (Difusión Térmica)

$$\text{Ecuación de calor en 1D: } \frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} \right)$$

$u(x, t)$ = temperatura, α = coeficiente de difusión



- Usando diferencias finitas, el espacio se discretiza en una cuadrícula (grid).
- Cada punto del espacio evoluciona basándose únicamente en sus vecinos inmediatos.
- Ventaja paralela: Podemos dividir la cuadrícula en bloques; cada hilo actualiza su bloque sin interferir con los demás.

Paralelizando el espacio: El bucle espacial

```
for(int t=0; t<Nt; t++){  
    #pragma omp parallel for
```

```
    for(int i=1; i<Nx-1; i++){  
        u_new[i] = u[i] + alpha*dt/(dx*dx)  
            * (u[i+1] - 2*u[i] + u[i-1]);  
    }
```

```
    u = u_new;
```

```
}
```

Se paraleliza exclusivamente el bucle espacial (i). El bucle temporal (t) debe permanecer secuencial porque el futuro depende del pasado absoluto.

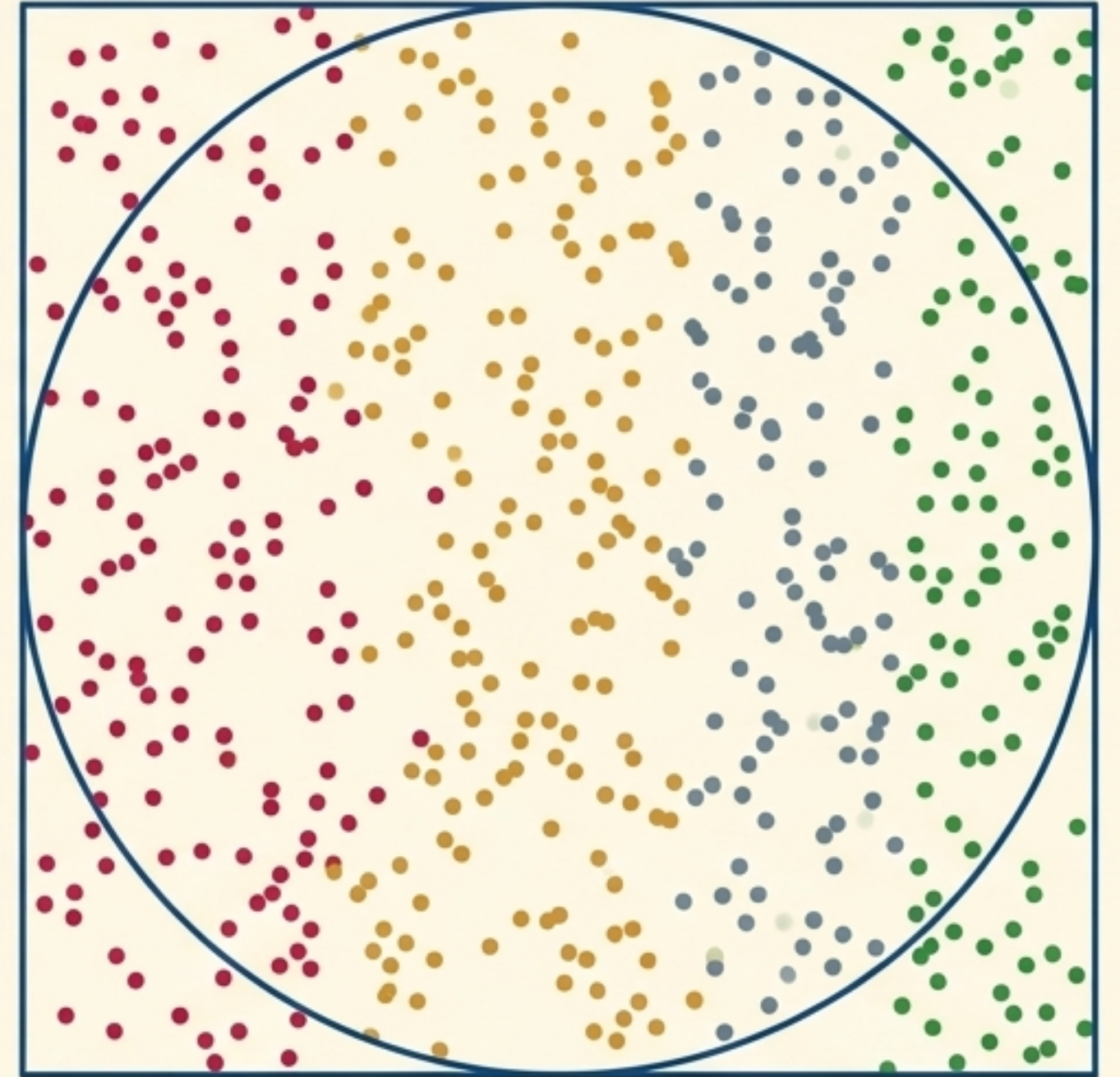
Paradigma 2: Paralelismo Estadístico (Monte Carlo para π)

$$\pi \approx 4 * \frac{N_{\text{dentro}}}{N_{\text{total}}}$$

- Generamos millones de puntos aleatorios en el cuadrado y contamos cuántos caen dentro del círculo.
- Paralelismo Perfecto: Cada punto es completamente independiente. No hay interacciones, no hay vecinos. Un escenario ideal para la máxima escalabilidad.

Demostración en Vivo

Ejecutar incrementando N de 10^4 a 10^8 , observando en un monitor de recursos cómo el uso del CPU salta al 100% en todos los núcleos.

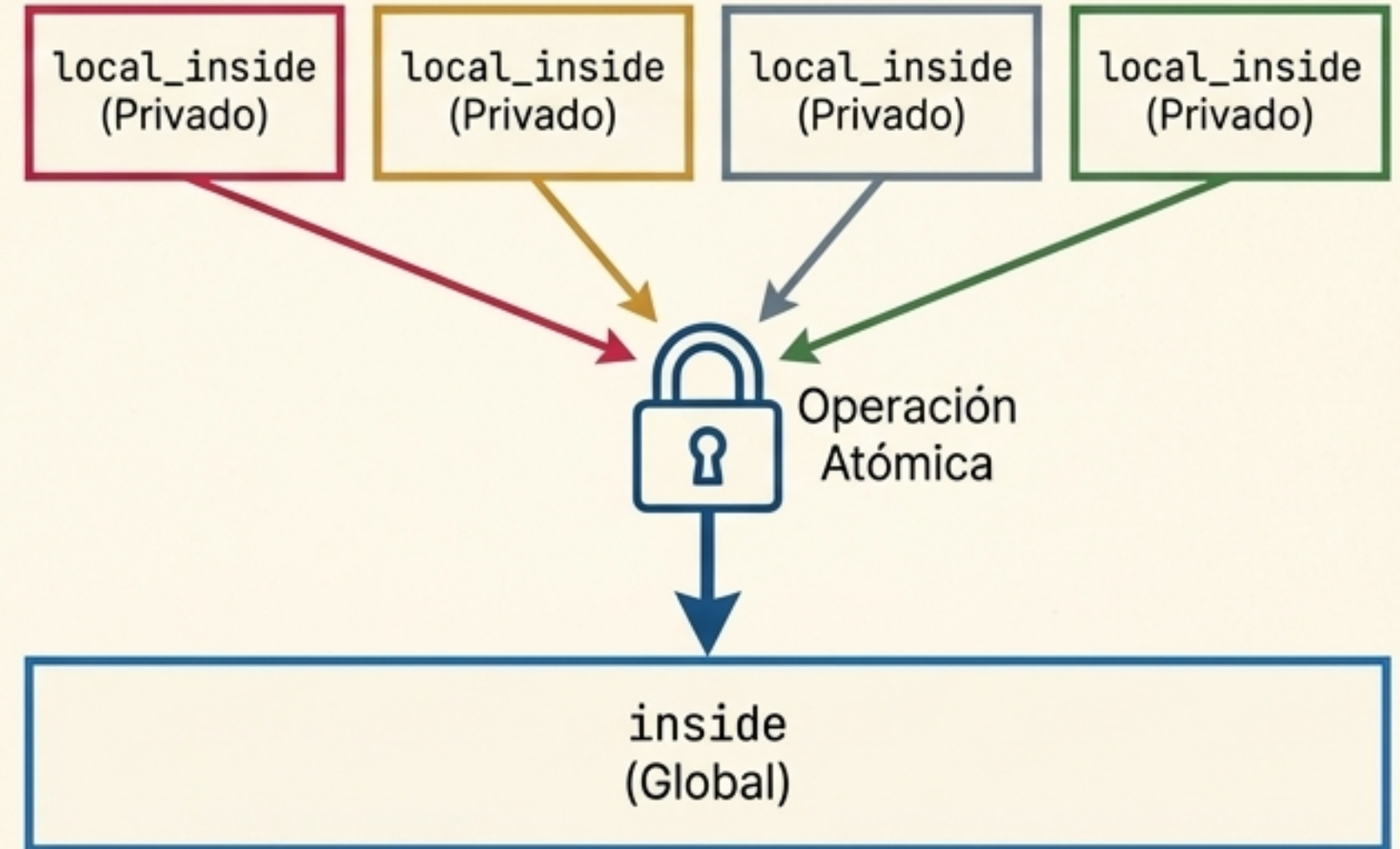


Sumando el caos: Variables atómicas

```
long inside = 0;
#pragma omp parallel
{
    long local_inside = 0;

    // Generación de x, y ...
    if(x*x + y*y <= 1) local_inside++;

    #pragma omp atomic
    inside += local_inside;
}
```

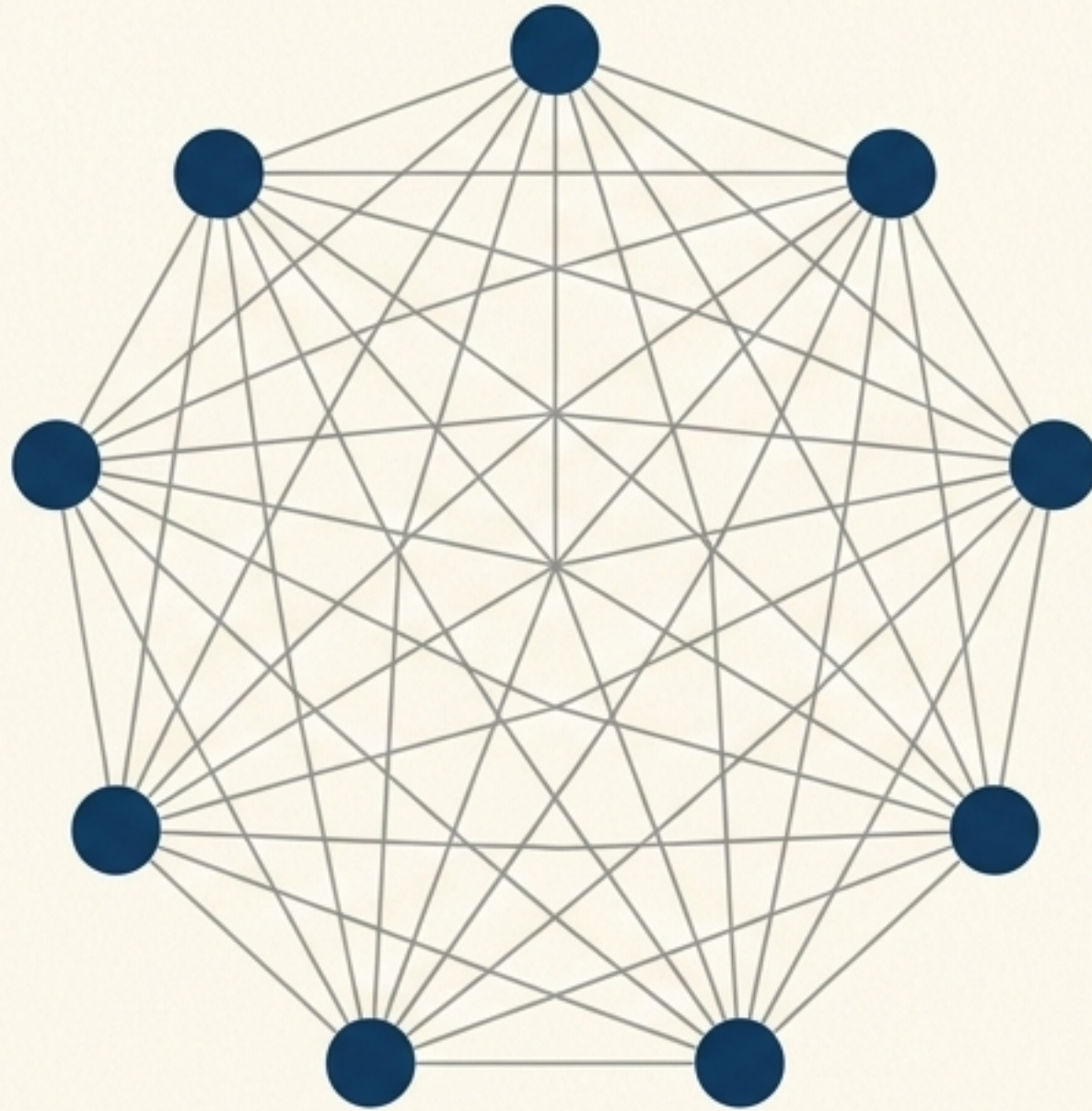


Paso 1: Cada hilo usa su propia variable `local_inside` para evitar colisiones constantes.

Paso 2: `#pragma omp atomic` asegura que la actualización de la variable global `inside` se haga hilo por hilo, protegiendo la memoria sin frenar el cálculo principal.

Paradigma 3: Paralelismo de Interacciones (N-Cuerpos Gravitacionales)

$$\text{Potencial: } V(r) = -G \sum_i^n \frac{m_i}{|r - r_i|} \quad \text{Fuerza: } F = G \frac{m_1 m_2}{r^2}$$



El desafío $O(N^2)$: Para N partículas, hay que calcular la interacción de cada una con todas las demás. Si duplicamos las partículas, el tiempo de cálculo se multiplica por cuatro. La paralelización no es opcional; es obligatoria.

Estrategia de anidación: El bucle exterior

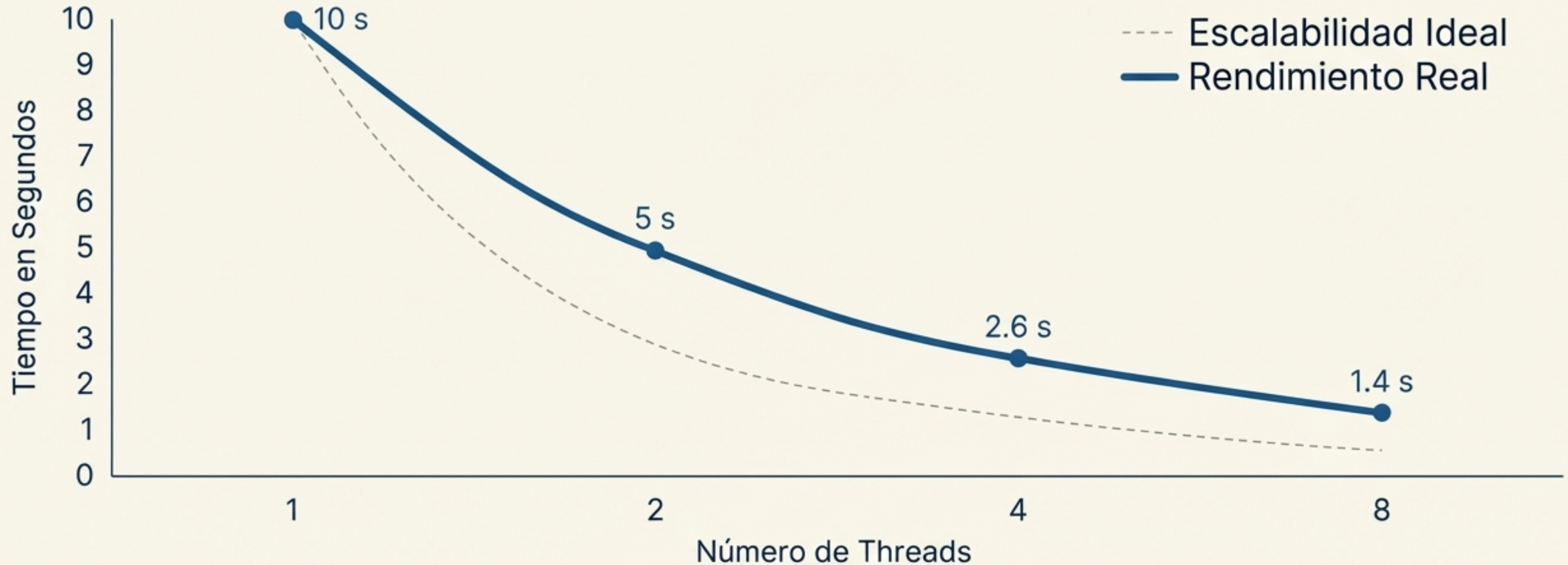
PARALELIZAR

```
#pragma omp parallel for
for(int i=0; i<N; i++){
    double fx=0, fy=0;
    for(int j=0; j<N; j++){
        if(i==j) continue;
        // Cálculo de fuerza ...
    }
    p[i].vx += fx*dt;
}
```

SECUENCIAL

Dividimos las partículas objetivo entre los núcleos (bucle i). Paralelizar el bucle interno (j) crearía demasiado *overhead* (costo administrativo para el CPU) al obligar al sistema a abrir y cerrar hilos millones de veces por segundo.

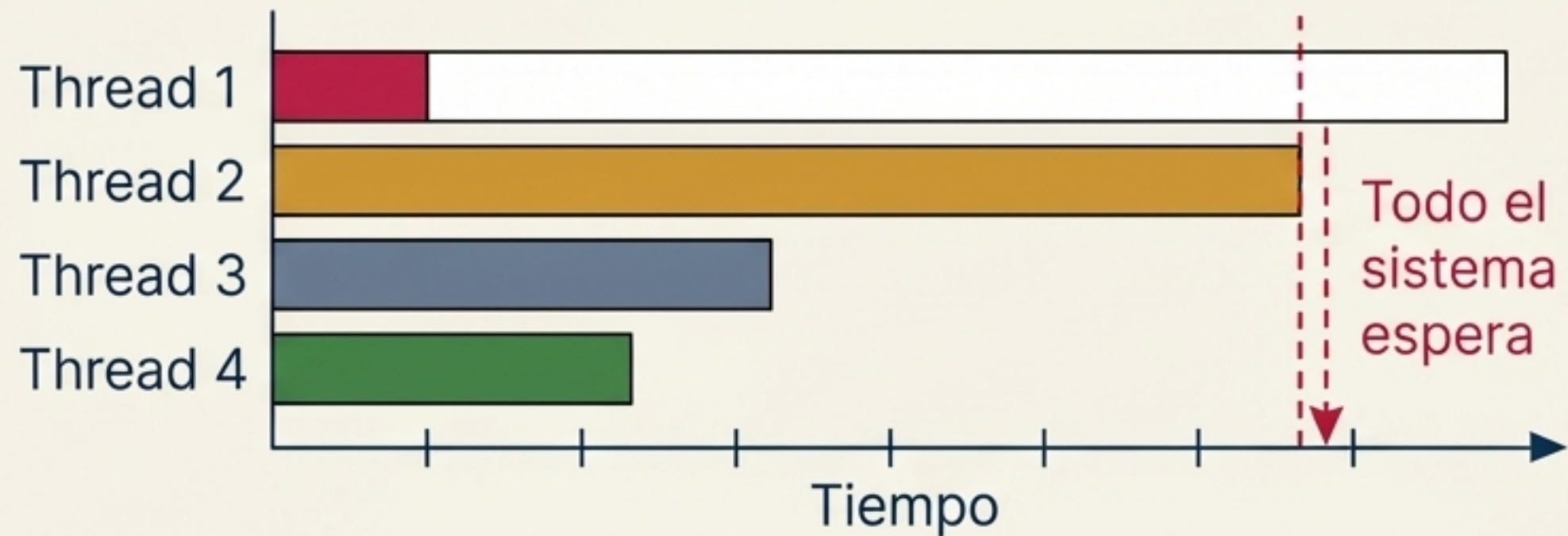
Realidad del Rendimiento: Speedup y Escalabilidad



La escalabilidad casi nunca es perfectamente lineal debido a la Ley de Amdahl. Crear hilos, dividir la memoria y sincronizar resultados consume tiempo. Sin embargo, reducir 10 segundos a 1.4 segundos transforma problemas intratables en simulaciones posibles.

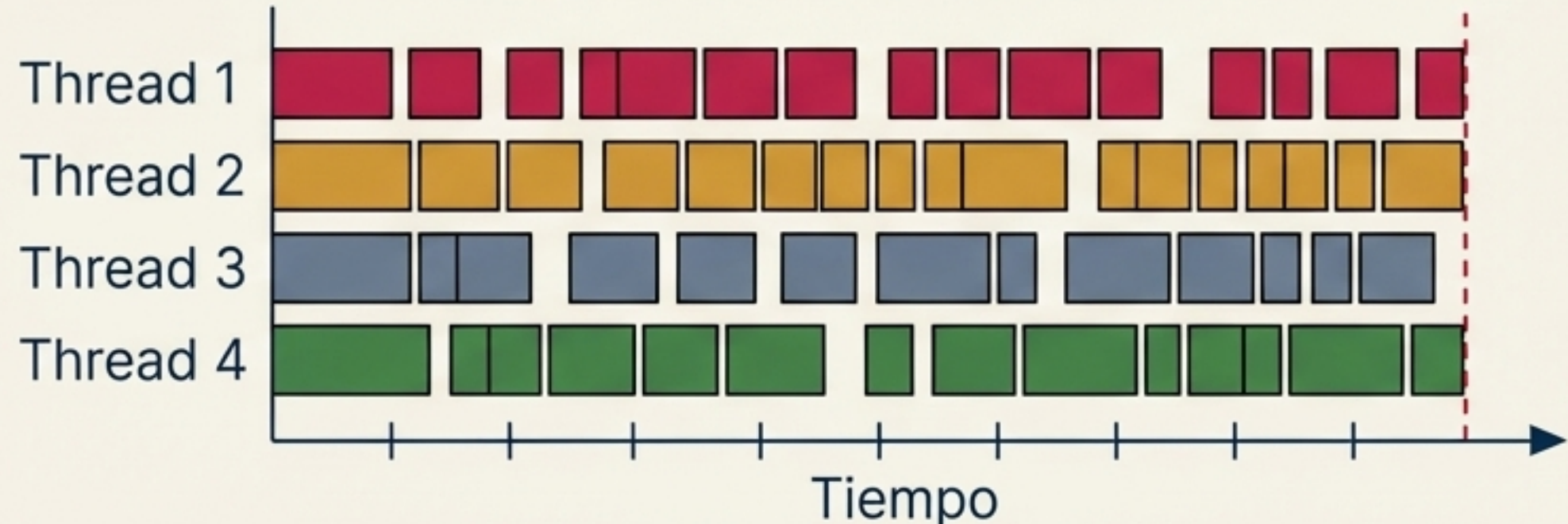
Trampas de HPC: El Balance de Carga

Balance Deficiente



En simulaciones irregulares (ej. galaxias agrupadas), algunos hilos terminan antes que otros y quedan inactivos.

Balance Óptimo con Planificación Dinámica

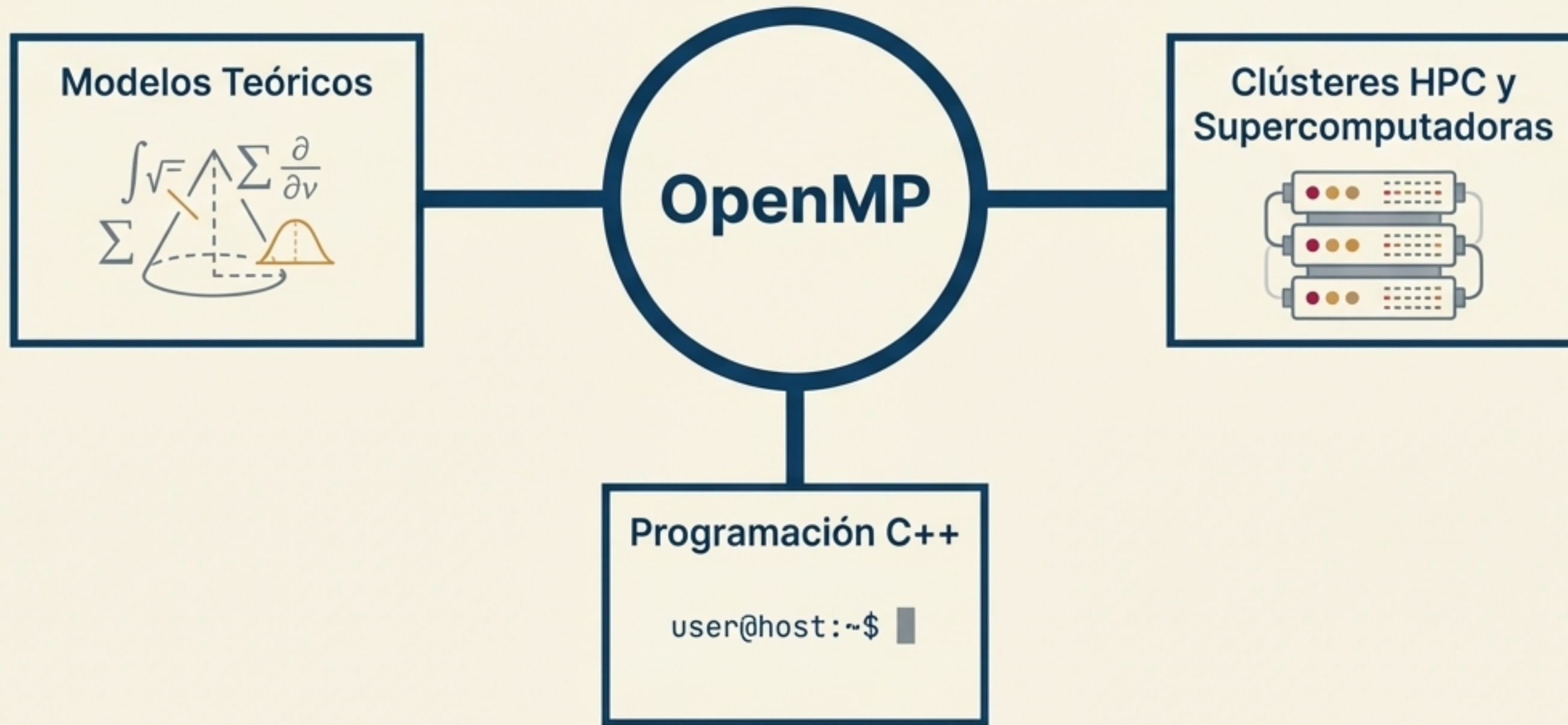


Solución avanzada: OpenMP permite planificación dinámica (`schedule(dynamic)`) para repartir el trabajo en pequeños bloques en tiempo real a los hilos desocupados.

Matriz de Síntesis: Eligiendo la estrategia correcta

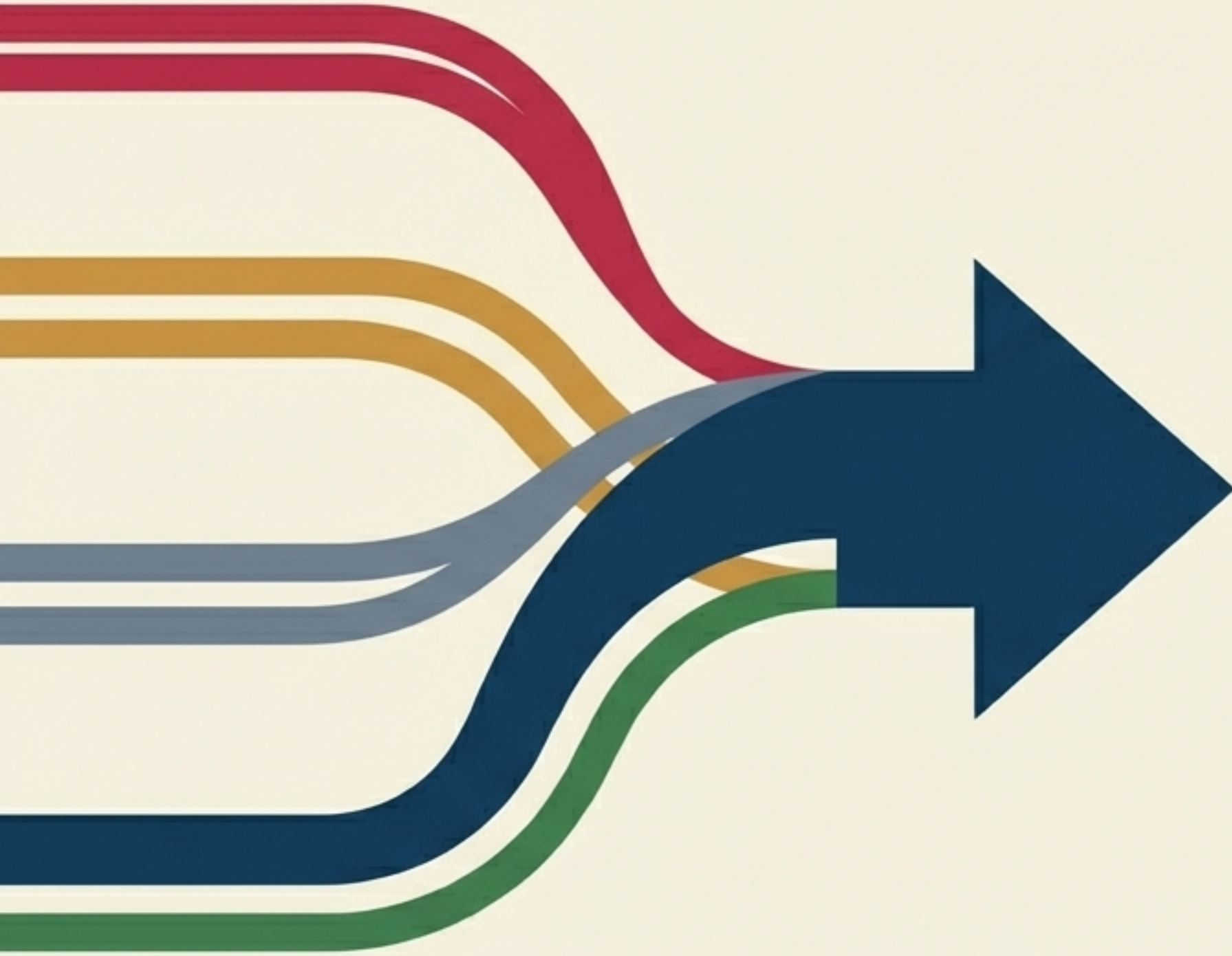
	Difusión (Ecuación de Calor)	Monte Carlo (Cálculo de π)	N-Cuerpos (Gravitación)
Tipo de Paralelismo	Espacial	Estadístico	Interacciones
Complejidad	$O(N)$	$O(N)$	$O(N^2)$
Solución de Memoria	Bloques Independientes	Operaciones Atómicas (atomic)	Bucle Exterior (reduction)
Aplicaciones HPC	Fluidos, Termodinámica	Cuántica, Física Estadística	Astrofísica, Dinámica Molecular

OpenMP como puente hacia la ciencia moderna



- OpenMP es el estándar de la industria para paralelización de memoria compartida (SMP).
- Es el primer paso crítico antes de pasar a sistemas distribuidos (MPI) o aceleración por hardware gráfico (CUDA / GPUs).
- Impulsa los descubrimientos actuales en ciencia de materiales, clima global y colisiones de partículas.

Conclusión y Siguietes Pasos



- La computación paralela es esencial para trascender los límites de la física secuencial.
- OpenMP permite paralelizar código C++ con mínimas modificaciones mediante directivas `#pragma`.
- Entender la naturaleza física del problema (espacio, estadística, interacción) dicta la estrategia computacional.

Desafío para el alumno: Implementar y medir el tiempo de ejecución del problema de N-Cuerpos en sus propias máquinas, modificando el número de hilos para comprobar empíricamente la Ley de Amdahl.